Sentinel
Autonomous SOAR Platform

Complete technical specification
SOAR + AI Agent + Pentest + GRC
For the Skyline managed security service

Graduation Project | Autonomous Security Operations Platform | October 2026

Contents

1. Vision, Positioning and Scope 3
2. System Architecture 4
3. Data Layer 5
4. Data Ingestion and Collection 6
5. Detection Engineering 7
6. Correlation, Incidents and Severity 8
7. Threat Intelligence and Enrichment 9
8. Playbooks and Orchestration 11
9. The Autonomous AI Agent (Tier-1 Replacement) 12
10. Response and Containment Engine 14
11. Case Management 15
12. Penetration Testing Module 16
13. Governance, Risk and Compliance (GRC) 17
14. Notifications and Escalation 19
15. Dashboard (Complete Specification) 20
16. Platform Security and Access Control 23
17. Observability and Reliability 24
18. Testing, Simulation and Validation 25
19. Key Performance Indicators 26
20. Advanced Features That Make the Project Outstanding 27
21. Coverage Against an Enterprise SOAR Reference, Opinion, and Sources 28

1. Vision, Positioning and Scope

1.1 What this platform is

Sentinel (working title) is an **autonomous Security Operations platform delivered as a managed service**. It combines three capabilities in one product: **continuous attack detection** (machine learning plus rules), **autonomous response** (an AI agent that performs the work of a SOC Tier-1 analyst and executes actions by itself), and **Governance, Risk and Compliance (GRC)** (assets, risks, controls, evidence and executive reporting). A built-in **penetration-testing module** continuously tests the customer and feeds its findings into both the SOAR and the GRC layers.

The customer in this project is **Skyline**, a company whose public web application (skyline-web) sits behind an HAProxy load balancer. We play the role of a security provider (an MSSP): we monitor Skyline, decide, act, and report.

> Design principle: this is not a miniature demo of a SOAR. It follows the structure of enterprise platforms such as Splunk SOAR and Cortex XSOAR (ingestion, correlation, case management, playbooks, orchestration, enrichment, response, dashboards, governance) and adds an LLM-based autonomous agent on top.

1.2 What replaces the Tier-1 analyst

A human Tier-1 analyst does five things: triage the alert, enrich it, decide true/false positive, contain the threat, and escalate or document. In Sentinel each of these is performed automatically by a specific component.

Tier-1 duty	Sentinel component	Result
Triage	Correlation engine + AI Triage Agent	Alerts grouped into incidents; verdict true/false positive
Enrichment	Enrichment service (VirusTotal, AbuseIPDB, GeoIP, MITRE, internal history)	Evidence bundle attached to every incident
Prioritisation	Severity scoring engine	Critical / High / Medium / Low with an explainable score
Containment	Response engine (HAProxy runtime API and others)	IP blocked, rate-limited or tarpitted within seconds
Documentation	AI Reporter Agent + case management	Incident report, timeline, MITRE mapping, lessons learned

1.3 Autonomy model (actions are executed automatically)

The agent **acts on its own**. No analyst has to click Approve for normal response actions. Safety does not come from a human gate; it comes from engineering controls around the agent:

- **Policy Guard** (plain code, not a prompt): allowlists, maximum blast radius, maximum action rate, maximum block duration.
- **Reversibility by default**: every containment action has a time-to-live and an automatic rollback.
- **Verification**: after every action the platform re-checks the target system to confirm the action took effect.
- **Confidence gates**: stronger actions require more independent evidence (see section 9.6).
- **Kill switch**: one switch that stops all autonomous execution instantly (the platform keeps detecting and recording).
- **Full audit trail** of every decision, tool call and action, with the reasoning text, so humans can review afterwards (post-hoc oversight).

1.4 Current status

Component	Status
ML attack classifier (Random Forest, 71 features, 7 classes)	Done
Customer website (skyline-web) and platform API	Done
Packet capture (cicflowmeter sniffer), rate-based detector, Telegram alerting, HAProxy	Working prototype (to be rebuilt on the new pipeline)
Everything else in this document	To be built

2. System Architecture

2.1 Layers

- **Sources layer**: network flows (cicflowmeter), web-server logs, HAProxy logs, application authentication events, pentest events, container and host logs.
- **Transport layer**: Redis Streams (durable, ordered, consumer groups, acknowledgements).
- **Detection layer**: ML inference service, rate-based rules, signature/rule engine (Sigma-style), log analytics rules.
- **Correlation layer**: groups alerts into incidents and opens cases.
- **Intelligence layer**: enrichment service (threat intelligence, geolocation, MITRE ATT&CK, internal history).
- **Decision layer**: AI agents plus the Policy Guard (the autonomous Tier-1).
- **Action layer**: playbook engine, automation executor, response connectors (HAProxy, website API, notifications).
- **Business layer**: GRC (assets, risks, controls, policies, evidence, reports).
- **Presentation layer**: the dashboard, the client portal, and the public REST API.

2.2 End-to-end data flow

```
sources -> [Redis Stream: events] -> detectors (ML / rate / rules)
        -> [Stream: alerts] -> correlation -> incident
        -> enrichment (VirusTotal, AbuseIPDB, GeoIP, MITRE, history)
        -> AI agent (verdict, severity, actions)  -> Policy Guard
        -> executor (HAProxy block, notify, case update) -> verification
        -> audit log + case timeline + dashboards + GRC risk update
```

2.3 Services (Docker Compose)

Service	Responsibility	Notes
skyline-web (mode A/B)	Customer application	MODE=vulnerable or secure via environment variable
haproxy	Reverse proxy and enforcement point	Runtime API socket enabled; blocklist ACL file
redis	Streams, caches, rate counters	Persistence (AOF) on
mongo	Primary datastore	Replica set optional
sniffer	cicflowmeter flow capture	Writes to Redis stream, not CSV
log-shipper	Tails web and HAProxy logs	Parses into the unified schema
ml-service	Model inference API	Loads model, scaler and feature list
detector	Consumes events, runs all detection layers	Horizontally scalable (consumer group)
correlator	Alerts to incidents and cases	Stateful windows in Redis
enrichment	Threat intel and context lookups	Quota governor and cache
agent	AI agents (LLM gateway, tools, memory)	Fallback decision engine included
executor	Playbooks and response actions	Idempotent, retried, audited
pentest-runner	Controlled security scans of the lab target	Scope-locked containers
api	REST API and WebSocket/SSE for live updates	JWT authentication
dashboard	Web UI	See section 15
reporter	Scheduled PDF reports	Executive and technical

2.4 Technology choices

- Python 3.11 for detectors, agent, executor and enrichment; FastAPI for services; pydantic for schemas.
- Redis Streams for messaging; MongoDB for documents; optional time-series collection for metrics.
- Dashboard: React with a charting library (preferred) or Streamlit multi-page (acceptable fallback).
- LLM access through a **gateway module** so the provider can be swapped (hosted API or a local model) and costs and latency are tracked in one place.

3. Data Layer

3.1 Unified event schema

Every source is translated into one schema before entering the pipeline. Without this, correlation and the agent cannot work across sources.

```
Event {
  event_id, ts (UTC), source (network|web|haproxy|auth|pentest|host),
  src_ip, dst_ip, src_port, dst_port, protocol,
  user, session_id, http_method, url, status, user_agent,
  event_type (flow|http_request|login_failed|login_ok|...),
  features{...flow features for the ML model...},
  raw (original text), tags[], tenant_id
}
```

3.2 Streams

Stream	Producer	Consumer
events	sniffer, log-shipper, app, pentest	detector (consumer group)
alerts	detector	correlator
incidents	correlator	enrichment, then agent
actions	agent, playbook engine	executor
audit	every service	audit writer (append-only)

- Use `XADD` with a capped length, `XREADGROUP` in consumer groups and `XACK` after processing, so a crashed worker never loses events.
- A **dead-letter stream** receives events that fail parsing or processing repeatedly.

3.3 MongoDB collections

Collection	Purpose	Key indexes
alerts	Every detection	ts, src_ip, severity, status
incidents	Correlated groups	status, severity, src_ip, opened_at
cases	Analyst/agent work items, notes, timeline	status, owner, sla_due
decisions	Agent verdicts with reasoning and tool calls	incident_id, ts
actions	Executed actions with result and rollback state	target, status, expires_at
blocklist / allowlist	Active blocks and protected addresses	ip (unique), expires_at (TTL)
ioc_cache	Threat-intel responses	indicator (unique), fetched_at (TTL)
ip_profiles	Memory per IP: counts, verdicts, first/last seen	ip
assets	GRC asset inventory	name, criticality
risks	Risk register	score, owner, status
controls / evidence / policies	GRC library	framework, control_id
findings	Pentest and vulnerability findings	severity, status, cwe
playbooks / runs	Definitions and executions	name, status
audit	Immutable audit trail	ts, actor
stats_hourly	Pre-aggregated metrics for the dashboard	hour, metric

3.4 Retention and privacy

Raw events 30 days, alerts and incidents 12 months, audit records and evidence per policy (suggest 24 months). Mask personal data (emails, usernames, tokens) in logs before they reach the agent or any external API.

4. Data Ingestion and Collection

4.1 Sources and methods

Source	Method	Events produced
Network flows	cicflowmeter on the monitored interface, writing to Redis Stream directly	flow records with 70+ features
skyline-web access logs	log-shipper tails the file or container output	http_request
HAProxy logs	stdout/syslog parsed with a regular expression	http_request with real client IP, status, timings
Application auth events	the website emits login success/failure events	login_failed, login_ok, signup, password_reset
Pentest runner	emits begin/end markers and scanner traffic tags	pentest_marker
Docker/host logs	optional agent	container_event
External systems	REST endpoint `POST /events` with an API key	any normalised event

4.2 Pipeline reliability features

- Parsing and normalisation with schema validation (pydantic); failed records go to the dead-letter stream.
- Deduplication on `event_id`; timestamp conversion to UTC; source tagging.
- Backpressure: if the stream length exceeds a threshold the log-shipper slows down and raises a platform alert.
- Collection-health dashboard: events per second per source, last-seen per source, lag per consumer group.
- **Real client IP**: enable `option forwardfor` in HAProxy and have the website trust the `X-Forwarded-For` header from the proxy only. Without it every event shows the proxy address and any IP block would hit the proxy itself.

5. Detection Engineering

5.1 Four detection layers

Layer	Input	Detects	Output
ML classifier	Network flow features	Benign, Bot, BruteForce, DDoS, DoS, Infiltration, Web attacks	label + class probabilities
Rate-based layer	Per-IP counters in a sliding window	Volumetric floods, request bursts, scan speed	rate alert with counters
Log rules	Web, HAProxy and auth events	Brute force, credential stuffing, SQLi/XSS patterns, directory scanning, abnormal 4xx/5xx	rule alert with matched evidence
Signature rules (Sigma-style YAML)	Any normalised event	Known attack patterns, scanner user-agents, webshell paths	rule alert with rule id and ATT&CK tag

5.2 ML service

- Wrap the trained model, scaler and feature list in an inference API (`POST /predict`) that returns the label, the full probability vector and the top contributing features.
- Feature alignment is critical: the names, order and scaling of features produced by the live sniffer must match the training columns exactly; add a unit test that compares both lists.
- Keep the probability, not only the label: the agent and the confidence gates use it.
- Monitor drift: store the distribution of key features and the predicted class mix per day and alert when they move.
- Evaluate per class (precision, recall, F1, confusion matrix) and keep the report with the model version.

5.3 Rule engine

- Rules are YAML files with an id, title, ATT&CK technique, condition, threshold, window, severity and false-positive notes.
- Rule types: single-event match, threshold (N events in T seconds per key), sequence (A then B), and absence.
- Examples: more than 5 failed logins per IP in 60 seconds; 20 or more 404 responses in 30 seconds; request URL containing union-select patterns; upload of an executable file type; a successful login directly after many failures from the same IP.
- Every rule has test events and an expected result; rules are version-controlled and deployed through a test stage.

5.4 Alert quality

- One alert is created per detection, never per packet; the correlator reduces noise later.
- Each alert carries engine, confidence, evidence and the raw event ids for traceability.
- False-positive feedback from the agent and analysts is stored per rule and per model so tuning has data.

6. Correlation, Incidents and Severity

6.1 Correlation methods

- **Entity grouping**: same source IP and attack family inside a time window are merged into one incident.
- **Sequence correlation**: reconnaissance, then exploitation, then post-exploitation by the same IP raises the incident to a multi-stage attack and increases severity.
- **Cross-source corroboration**: a network ML alert plus a web-log rule alert for the same IP gives higher confidence than either alone.
- **Time windows**: sliding five-minute windows with late-event tolerance; incidents close after a quiet period.
- Window state is stored in Redis with expiry so correlators can be restarted safely.

6.2 Incident record

Incident id, entities (IPs, users, URLs), member alerts, first/last seen, attack families, ATT&CK techniques, affected assets, severity, status, and links to the decision, actions and case.

6.3 Severity scoring

```
score = 0.30 * attack_type_weight      (DDoS/web-exploit high, scan low)
      + 0.20 * detection_confidence    (ML probability, engine agreement)
      + 0.20 * asset_criticality       (from the GRC asset inventory, 1-5 normalised)
      + 0.15 * ip_reputation           (VirusTotal / AbuseIPDB / internal history)
      + 0.10 * recurrence              (how often this IP returned)
      + 0.05 * multi_stage_bonus       (sequence correlation hit)
Critical >= 0.80 | High 0.60-0.79 | Medium 0.40-0.59 | Low < 0.40
```

The weights live in configuration, and the score is stored together with its components so the dashboard can show **why** an incident is Critical.

7. Threat Intelligence and Enrichment

7.1 Enrichment pipeline

When an incident opens, the enrichment service gathers context in parallel, caches it, and attaches one evidence bundle: IP reputation, geolocation and ASN, ATT&CK mapping, history of this IP in our own data, asset context for the target, and any related findings from the pentest module.

7.2 VirusTotal integration (details)

- **API**: VirusTotal API v3. Authentication is an API key sent in the `x-apikey` request header.
- **Endpoints used**: `GET /api/v3/ip_addresses/{ip}`, `GET /api/v3/domains/{domain}`, `GET /api/v3/urls/{id}` and `GET /api/v3/files/{hash}` (for uploaded files on the website).
- **Fields extracted** from the response attributes: `last_analysis_stats` (counts of malicious, suspicious, harmless, undetected), `reputation`, `as_owner`, `asn`, `country`, `tags`, `last_analysis_results` (per-engine verdicts) and related communicating files when needed.
- **Rate limits**: the free public key allows about 4 requests per minute and 500 per day (premium keys have no such limit). The platform must therefore include a **quota governor**: a token-bucket limiter, a priority queue (Critical incidents first), and a cache.
- **Caching**: store each answer in `ioc_cache` with a TTL (for example 24 hours for IPs); never query the same indicator twice inside the TTL.
- **Private and lab addresses** (10.x, 172.16-31.x, 192.168.x, loopback) are never sent to VirusTotal; they are resolved from internal history only.
- **Degraded mode**: if the quota is exhausted or the service is down, the incident continues with the other sources and the evidence bundle records that VirusTotal was unavailable.
- **File analysis**: files uploaded to skyline-web are hashed (SHA-256) and looked up by hash; only unknown hashes are optionally submitted, because uploaded files may contain private data.
- **Derived score**: `vt_score = malicious / total_engines`, combined with reputation into the single `ip_reputation` input of the severity formula.

7.3 Other intelligence sources

Source	Used for	Notes
AbuseIPDB	Crowd-sourced abuse confidence score per IP	Free tier has a daily quota; cache results
GeoIP database (MaxMind GeoLite2)	Country, city, ASN, map coordinates	Local database; no API quota
Internal reputation	Our own history per IP	The strongest signal for repeat offenders
Open feeds (optional)	Known-bad IP and domain lists	Imported on a schedule with expiry
Pentest findings	Whether the targeted endpoint is known to be vulnerable	Raises severity when an attack matches a known weakness

7.4 MITRE ATT&CK integration (details)

- **Data source**: the ATT&CK dataset published as STIX 2.1 JSON in the `mitre-attack/attack-stix-data` repository (Enterprise collection). The current release at the time of writing is **ATT&CK v19 (April 2026)**. Load it at start-up (the `mitreattack-python` library helps) and refresh it on a schedule.
- **Important v19 change**: the Enterprise tactic *Defense Evasion* was split into **Stealth** and **Defense Impairment**. Therefore never hard-code tactic names; always read the tactic of a technique from the loaded STIX data.
- **Mapping table** (reviewed by the agent, stored as data, not code):

Detected behaviour	Technique	Tactic (typical)
Login brute force	T1110 (T1110.001 guessing, .003 spraying, .004 credential stuffing)	Credential Access
SQL injection / exploit of the web app	T1190 Exploit Public-Facing Application	Initial Access
Network flood (DDoS)	T1498 Network Denial of Service	Impact
Application-layer flood (DoS)	T1499 Endpoint Denial of Service	Impact
Port and service scanning	T1046 Network Service Discovery, T1595 Active Scanning	Discovery, Reconnaissance

Detected behaviour	Technique	Tactic (typical)
Malicious upload / webshell	T1505.003 Web Shell	Persistence
Botnet beaconing	T1071 Application Layer Protocol	Command and Control
Use of stolen credentials	T1078 Valid Accounts	Initial Access / Persistence

- **Outputs**: technique ids on every alert and incident, an ATT&CK coverage heatmap (which techniques we detect, which we do not), and an exported **ATT&CK Navigator layer** (JSON) per incident and per period.
- **Coverage-gap analysis**: techniques with no detection rule or model class are listed so the team can prioritise new detections.

7.5 Indicator management

Indicators (IP, domain, URL, hash) are normalised, deduplicated, scored with confidence, and expire (freshness). Each indicator links to the incidents where it appeared, and can be exported in STIX format.

8. Playbooks and Orchestration

8.1 Playbook engine

Playbooks are **YAML definitions** interpreted by the engine, so behaviour changes without code changes. A trigger starts a run; steps execute in order or in parallel; every run is stored with inputs, outputs and timings.

```
name: web_bruteforce_response
trigger: {incident_type: brute_force, min_severity: medium}
steps:
  - id: enrich      action: enrichment.collect
  - id: decide      action: agent.decide        # AI agent returns verdict + actions
  - id: contain     action: haproxy.block_ip    when: decide.verdict == "true_positive"
                    args: {ttl_minutes: "{{ decide.ttl }}"}
  - id: lock        action: webapp.lock_account when: decide.target_user
  - id: notify      action: notify.send         args: {channel: telegram}
  - id: verify      action: haproxy.verify_block
on_error: {retry: 3, backoff: exponential, fallback: notify.send}
```

8.2 Engine capabilities

- Conditions, branching, loops with limits, parallel branches, sub-playbooks, variables and templates.
- Execution control: pause, resume, cancel, timeouts, retries with backoff, checkpoints, and **idempotency keys** so a retried step never blocks an IP twice.
- Lifecycle: draft, test, publish, version, roll back, retire; dry-run mode that logs what would happen without acting; mock connectors for tests.
- Safety: loop and step limits, duplicate-run protection, emergency stop (kill switch).

8.3 Playbook library to ship

Playbook	Trigger	Main actions
Brute force / credential stuffing	Repeated failed logins	Block IP, lock or flag account, force reset, notify
DoS / DDoS flood	Rate or ML DoS/DDoS alert	Rate-limit, tarpit, then block; protect upstream capacity
SQL injection / XSS / web exploit	Signature or ML Web alert	Block IP, add WAF rule, flag vulnerable endpoint to GRC
Port scan / reconnaissance	Scan rule	Tarpit, short block, watch list
Malicious file upload	Upload rule / hash reputation	Quarantine file, block IP, notify
Botnet / beaconing	Bot class	Block, isolate container if internal
Authorised pentest	Pentest marker	Tag, do not block scanner, verify detection fired
Repeat offender	IP seen N times	Escalate block duration (1h, 6h, 24h)
False-positive handling	Agent verdict false_positive	Close, record feedback, tune rule
Service health	Platform alert	Notify, restart service, fall back to rules

9. The Autonomous AI Agent (Tier-1 Replacement)

9.1 Agent roles (multi-agent design)

Agent	Job	Tools it may use
Triage Agent	Reads the incident and evidence, decides verdict and severity	Read-only lookup tools
Investigator Agent	Collects extra evidence when the verdict is uncertain	Search events, IP history, VirusTotal, pentest findings
Responder Agent	Chooses and executes containment actions	Response tools (through the Policy Guard)
Reporter Agent	Writes the incident report, timeline and lessons learned	Case and audit read tools
Tuning Agent	Proposes rule thresholds and allowlist changes from feedback	Rule repository (proposal mode)
Pentest Agent	Plans scans of the lab target and summarises findings	Scope-locked scanner tools
GRC Agent	Updates risk scores and drafts the executive summary	GRC read/write tools

9.2 Internal components

- **Context builder** (plain code): assembles a compact JSON of the incident, alerts, evidence, asset, history and relevant policies; redacts personal data; truncates attacker-controlled strings.
- **LLM gateway**: provider routing, timeouts, retries, token and cost accounting, model fallback.
- **Prompt library**: versioned system prompts, few-shot examples per attack type, and regression tests.
- **Tool registry**: each tool has a name, description, JSON schema for arguments, a permission scope, and a validator.
- **Reasoning loop**: bounded to a maximum number of steps and seconds; each step is a tool call or a final decision.
- **Output validator**: the final answer must match a strict JSON schema or it is rejected and retried.
- **Policy Guard and Executor**: checks and executes actions (section 10).
- **Memory**: per-IP profile (first seen, counts, past verdicts and actions) plus a store of analyst corrections used as examples in later prompts.
- **Fallback decision engine**: deterministic rules that take over when the LLM is unavailable, slow or returns invalid output; the decision records which path was used.

9.3 Decision output schema

```
{
 "verdict": "true_positive | false_positive | uncertain",
 "severity": "critical | high | medium | low",
 "confidence": 0.0-1.0,
 "evidence": ["ml:DoS p=0.97", "vt:malicious=12/90", "rate:203 req/10s"],
 "mitre": ["T1498"],
 "actions": [{"tool": "block_ip", "args": {"ip": "...", "ttl_minutes": 60}}],
 "reasoning": "five sentences maximum, citing evidence ids"
}
```

9.4 Tool catalogue

Tool	Type	Description
get_incident / get_alerts	read	Fetch incident details and member alerts
ip_history	read	Prior sightings, verdicts and actions for an IP
vt_lookup	read	VirusTotal summary (through quota governor and cache)
abuseipdb_check	read	Abuse confidence score
geo_lookup	read	Country, ASN, coordinates
mitre_lookup	read	Technique name, tactic, mitigations, detections
asset_info	read	Criticality, owner, business service of the target

Tool	Type	Description
search_events	read	Query recent events by IP, URL or user
pentest_findings	read	Known weaknesses of the targeted endpoint
block_ip	write	Block at HAProxy with a TTL
rate_limit_ip / tarpit_ip	write	Softer containment
lock_account	write	Lock or flag a website account through the API
add_waf_rule	write	Add a pattern block for a specific attack
notify	write	Telegram, email or webhook
update_case	write	Add notes, change status and fields
propose_rule_change	write	Create a tuning proposal (reviewed or auto-applied by policy)

9.5 Prompt design

- **System prompt** defines the role (Tier-1 analyst for Skyline), the evidence-first procedure, the output schema, and the hard rules: only use provided tools, never invent evidence, answer uncertain when evidence conflicts.
- **Untrusted data rule**: everything copied from logs (URLs, user-agents, headers, payloads) is wrapped in a delimiter and declared to be data, never instructions.
- **Few-shot examples**: at least three per attack family, including one false positive and one uncertain case.
- **Procedure inside the prompt**: (1) restate the incident, (2) list what is known, (3) call tools to fill gaps, (4) weigh evidence for and against, (5) output the JSON decision.
- Prompts are versioned; each decision stores the prompt version and model id for reproducibility.

9.6 Confidence gates (how autonomy is controlled)

Gate	Condition	Actions the agent may execute by itself
G0	Any incident	Record, enrich, notify
G1	verdict true_positive and confidence >= 0.60	Rate-limit, tarpit, short block (up to 15 minutes)
G2	confidence >= 0.80 and at least two independent evidence types	Block up to 6 hours, lock account, add WAF rule
G3	confidence >= 0.90, two evidence types, and severity Critical or repeat offender	Block up to 24 hours, isolate container, broad rules
Uncertain	verdict uncertain or evidence conflicts	Investigator Agent collects more evidence; only G0 actions until resolved; case flagged for later human review

9.7 Defending the agent itself

- **Prompt injection**: attackers control parts of the logs. Four layers: delimiting and truncating untrusted text, explicit instructions in the prompt, schema-validated output, and a Policy Guard in code that rejects any forbidden action no matter what the model says.
- **Tool abuse**: each agent only receives the tools of its role; arguments are validated; call counts and rates are limited.
- **Data leakage**: personal data is masked before leaving the platform; the LLM never receives secrets.
- **Red-team tests**: a test suite sends crafted requests (for example a user-agent containing an instruction to whitelist the sender) and asserts that no allowlist change or harmful action occurs.

9.8 Evaluating the agent

- Golden set of at least 50 incidents with a human-labelled correct verdict and action.
- Metrics: verdict agreement, wrong-action rate (target zero for destructive actions), average decision time, tool calls per decision, tokens and cost per incident, fallback rate, injection resistance.
- Baseline comparison: the same incidents handled by the fixed fallback engine; if the agent is not clearly better the project should say so honestly.

10. Response and Containment Engine

10.1 Enforcement points

Point	Actions	Mechanism
HAProxy (edge)	Block, rate-limit, tarpit an IP	ACL file plus runtime API; stick tables
skyline-web (application)	Lock or flag an account, revoke sessions, add a WAF pattern	Authenticated platform API on the website
Containers	Disconnect a compromised container from the network, stop it	Docker API (restricted socket)
Notifications	Telegram, email, webhook	Notification engine

10.2 Automatic IP blocking with HAProxy (how it works)

HAProxy exposes a **Runtime API** through a `stats socket`. Entries can be added to or removed from an ACL while HAProxy is running, with no restart and no dropped connections. The commands used are `add acl`, `del acl`, `show acl` and `clear acl`; multi-change transactions use `prepare acl` and `commit acl`.

```
global
    stats socket /var/run/haproxy.sock mode 660 level admin
frontend web_in
    bind *:80
    option forwardfor
    acl blocked src -f /etc/haproxy/blocklist.acl
    http-request deny deny_status 403 if blocked
```

```
# executor (conceptual)
echo "add acl /etc/haproxy/blocklist.acl 203.0.113.7" | socat stdio /var/run/haproxy.sock
echo "del acl /etc/haproxy/blocklist.acl 203.0.113.7" | socat stdio /var/run/haproxy.sock   # at TTL expiry
echo "show acl /etc/haproxy/blocklist.acl" | socat stdio /var/run/haproxy.sock               # verification
```

- Every block is also written to the blocklist file so that it survives a reload or restart; the executor reconciles the file, the live ACL and the database.
- **Time-to-live**: each block stores `expires_at`; a scheduler issues `del acl` at expiry. Repeat offenders get escalating durations (for example 1 hour, 6 hours, 24 hours).
- **Verification**: after `add acl`, the executor runs `show acl` (and optionally a test request from a canary client) to confirm the entry is active, and records the result.
- **Docker Desktop caveat**: when browsing from the host machine, HAProxy may see the Docker gateway address instead of the real client. Use a dedicated attacker container with its own address on the compose network for demonstrations, so that blocking the attacker never blocks the administrator.

10.3 Pre-agent protection (always on)

Stick tables in HAProxy track requests per IP and reject floods immediately (HTTP 429) before any AI is involved; this keeps the platform alive during large floods while the agent decides on longer blocks. The `tarpit` action slows abusive clients without a hard block.

10.4 Policy Guard (code that the agent cannot override)

- **Allowlist** of addresses that can never be blocked (administrators, DNS, health checks, the load balancer, the pentest runner when a scan is authorised).
- **Maximum block duration** per gate (section 9.6) and a global cap.
- **Blast-radius limits**: no CIDR ranges unless gate G3; maximum number of active blocks; maximum blocks per minute (a runaway-protection circuit breaker that pauses autonomy and alerts).
- **Scope checks**: actions only apply to Skyline assets listed in the asset inventory.
- **Idempotency and rollback**: each action stores an inverse operation and runs it at expiry or when a case is closed as a false positive.
- **Kill switch**: global and per-playbook; honoured by every executor worker before each action.

10.5 Action record

Every action stores: id, incident id, decision id, actor (agent or playbook), tool, arguments, gate, result, verification result, expiry, rollback status and timestamps. These records appear in the dashboard **Autonomous Actions Ledger**, where an analyst can see and reverse any action with one click.

11. Case Management

- A case is opened automatically for every incident above Low severity, and for every uncertain decision.
- Fields: id, title, severity, status, owner (agent, analyst or team), affected assets and identities, related incidents, indicators, MITRE techniques, SLA timers, evidence, notes and timeline.
- States: New, Triaged, Contained, Investigating, Pending Review, Resolved, Closed, False Positive, Reopened.
- The agent writes the first summary and updates the timeline for every enrichment, decision and action.
- Human analysts can comment, reassign, override a verdict, reverse an action, add evidence and close with a root cause and lessons learned.
- SLA tracking: time to acknowledge, time to contain, time to resolve; breach warnings in the dashboard and by notification.
- Metrics produced: MTTD, MTTA, MTTC (contain), MTTR, reopen rate, false-positive rate, percentage handled fully autonomously.

12. Penetration Testing Module

12.1 Purpose

The module tests Skyline continuously and on demand, produces findings that feed the GRC risk register, and **proves that detection and response work** by generating real attack traffic that the SOAR must catch.

> Safety rule: scans run only against the lab target and only with the scope file loaded. The code (not the prompt) refuses any target outside `scope.yaml`.

12.2 Two versions of the customer website

Mode	Description	Purpose
A: vulnerable	Deliberate weaknesses: string-built SQL in search, unescaped comments (XSS), weak password storage, unrestricted uploads, missing security headers, no rate limiting, verbose errors	Gives the pentest module real findings and gives the SOAR real attacks to detect
B: hardened	Parameterised queries, output escaping, strong password hashing, upload validation, rate limiting, security headers, CSRF protection	Demonstrates remediation and the before/after improvement

Both run from the same code base; an environment variable selects the mode. Both stay inside an isolated Docker network.

12.3 Tools orchestrated (all against the lab target)

Tool	Used for
Nmap	Service and version discovery, open ports
OWASP ZAP (baseline and active scan)	Web vulnerabilities, headers, cookies, injection and XSS probes
Nikto	Common server misconfigurations and files
sqlmap	Confirms SQL injection on selected endpoints
Hydra	Tests password policy with a small wordlist on the login form
Nuclei (optional)	Template-based checks for known issues

12.4 Pipeline

- **Planner** (the Pentest Agent): reads the scope and the site map, chooses scans and orders them.
- **Runner**: each tool runs in its own container with CPU, time and network limits; output is saved.
- **Parsers**: convert Nmap XML and ZAP/Nikto/sqlmap output into one finding format (title, endpoint, evidence, CWE, OWASP category, CVSS, status).
- **Scoring**: CVSS base score (v3.1 or v4.0) plus the asset criticality gives a priority.
- **Report**: executive summary, technical details, reproduction steps and recommended fixes, produced as PDF.
- **Retest**: after switching to mode B, the same scans run again and the report shows fixed, remaining and new findings.

12.5 Integration with the rest of the platform

- Pentest traffic carries a marker and comes from a known address; the SOAR must **detect it** (this is recorded as a detection test) but does not block the runner and labels the incidents `authorized_test`, so statistics stay honest.
- Findings become entries in the risk register and remediation tracker (section 13).
- A real attack against an endpoint that has an open finding gets a severity bonus in the scoring formula.
- A **detection coverage report** compares each attack type launched with what the platform detected, mapped to ATT&CK techniques.

13. Governance, Risk and Compliance (GRC)

13.1 Asset inventory

- Assets: web application, load balancer, database, containers, API, the SOAR components themselves, and data stores.
- Fields: id, name, type, owner, business service, **criticality (1-5)**, data classification (public, internal, confidential), exposure (internet-facing or internal), environment, last-seen, status.
- Assets link to alerts, incidents, findings, risks and controls; asset criticality feeds severity scoring.

13.2 Risk register

- Each risk: id, title, description, asset, threat, vulnerability, owner, **likelihood (1-5)**, **impact (1-5)**, inherent score, existing controls, residual score, treatment (mitigate, accept, transfer, avoid), due date, review date, status.
- Score = likelihood x impact (1-25): 1-4 Low, 5-9 Medium, 10-16 High, 17-25 Critical.
- **Live risk updates**: likelihood is recalculated from real data (number of incidents against the asset, open findings, exploitability); impact comes from asset criticality and data classification. The risk moves when attacks and findings move.
- Heatmap 5x5 and trend lines (risk over time, residual versus inherent).
- Risk acceptance requires an owner, a reason and an expiry date; expired acceptances reappear automatically.

13.3 Control library and framework mapping

Framework	How it is used
NIST Cybersecurity Framework 2.0	Functions Govern, Identify, Protect, Detect, Respond, Recover; the platform shows coverage per function
ISO/IEC 27001:2022 Annex A	93 controls in four themes; map relevant controls, for example inventory of assets (5.9), incident management (5.24 to 5.28), secure authentication (8.5), management of technical vulnerabilities (8.8), logging (8.15), monitoring activities (8.16)
CIS Critical Security Controls v8	Safeguard-level checklist for the web stack
OWASP Top 10	Maps pentest findings and attack types to categories such as injection, broken authentication and security misconfiguration
MITRE ATT&CK	Links detections and incidents to techniques (section 7.4)
Personal data protection law (optional)	Breach-notification and data-handling checklist for personal data held by the customer; confirm the legal text with a legal advisor

- Every control has: id, framework, description, owner, implementation status (Implemented, Partial, Not implemented, Not applicable), maturity (0-5), linked evidence, linked tests, review date.
- Verify control numbers against the official standard text before using them in the final report.

13.4 Compliance scoring

```
control_score  = Implemented 1.0 | Partial 0.5 | Not implemented 0.0   (N/A excluded)
framework_pct  = sum(control_score x weight) / sum(weight) x 100
```

The compliance centre shows percent per framework and per function, the top gaps, and which gaps would be closed by finishing which remediation task.

13.5 Automatic evidence collection

- Evidence is gathered by the platform itself: detection rules active, response actions executed (with timestamps), scan reports, audit logs, access reviews, backup tests, configuration snapshots.
- Each item has a hash, a timestamp, the source and the control(s) it supports, which gives auditors a verifiable trail (chain of custody).
- Controls with no fresh evidence inside their review period are flagged stale.

13.6 Policy management

- Policy library with owner, version, approval status, review date and acknowledgement tracking.

- **Policies that execute**: some policies are linked to playbooks, so governance becomes enforcement. Example: the policy "brute-force sources are blocked for 24 hours after repeated attempts" is the same object that parameterises the brute-force playbook.
- Policy exceptions with expiry and approver.

13.7 Remediation tracking

Findings and control gaps create tasks with owner, due date, priority and status. A task closes only when the retest confirms the fix. Overdue tasks raise the related risk score.

13.8 Key Risk Indicators and reports

- KRIs: open critical findings, mean time to remediate, patch/finding ageing, percent of incidents handled autonomously, detection coverage of ATT&CK techniques, number of expired exceptions, policy review overdue count.
- Reports: **Executive report** (one to two pages: risk posture, top risks, compliance percentage, incidents summary, decisions needed) and **Technical report** (incidents, findings, actions, evidence), generated as PDF, on demand and weekly.
- **Audit view**: read-only access for an auditor role, showing controls, evidence and the immutable audit log.

14. Notifications and Escalation

Channel	Used for
Telegram	Real-time alerts, action notices (what the agent did and why), commands
Email	Critical incidents, daily and weekly digests, reports
Webhook	Integration with external ticketing or chat tools
Dashboard	Live feed and notification centre

- Routing rules by severity, asset and time; templates with the evidence summary and a link to the case.
- Notices describe actions **after** they happen (autonomous model): what was blocked, why, for how long, and how to reverse it.
- Deduplication and quiet-period rules to avoid floods of messages; delivery tracking and retries.
- Escalation timers: a case that stays uncontained or unreviewed beyond its SLA escalates to the next channel or person.
- Optional **ChatOps** over Telegram: `/status`, `/incident <id>`, `/unblock <ip>`, `/killswitch on|off`, with authentication of authorised users.

15. Dashboard (Complete Specification)

15.1 Technology and behaviour

- **Frontend**: React (recommended) with a charting library, a map component for the attack map, a graph component for entity relationships, and virtualised tables for large lists. If the team prefers Streamlit, use multi-page mode with caching and server-side pagination, accepting slower interaction.
- **Backend**: FastAPI REST endpoints plus **Server-Sent Events or WebSocket** for live updates (new alerts, incident changes, actions) with no manual refresh.
- **Performance rules**: pre-aggregated collection `stats_hourly`, indexes on time, IP, severity and status, server-side pagination and filtering, a cache for expensive widgets, and a maximum range for raw-event queries.
- **Design rules**: dark theme by default with a light option; fixed severity colours (Critical red, High orange, Medium yellow, Low blue); every number is clickable and drills down to the underlying records; every table can export CSV; every page can be shared as a link with its filters.

15.2 Global elements (on every page)

- Top bar: global search (IP, incident, case, asset, CVE, technique), time-range picker (live, 1h, 24h, 7d, 30d, custom), notification bell, autonomy-state badge (Autonomous / Shadow / Paused), kill-switch button (confirmation, admins only), user menu.
- Left navigation grouped as Operations, Intelligence, Pentest, GRC, Platform and Admin; items hidden when the role has no access.
- Connection indicator for the live stream; banner when a data source is degraded.

15.3 Operations pages

Page 1 - Command Center (home)

- KPI tiles: open incidents by severity, alerts in 24h, percent handled autonomously, MTTD, MTTR, active blocks, agent accuracy, platform health.
- Charts: alerts per hour stacked by severity; attack-type distribution (donut); top 10 attacking IPs (bar); top targeted endpoints; engine contribution (ML, rate, rules, signatures).
- Live feed of the last events and agent actions, with severity badges.
- Mini-map of attack origins; ATT&CK heat strip; SLA-at-risk list.

Page 2 - Live Threat Map and Feed

- World map with animated attack arcs from source country to Skyline; filter by attack type and severity; click a country for its IPs.
- Side panel: newest incidents; hovering an arc shows IP, reputation, technique, and the action taken.

Page 3 - Alerts Explorer

- Table: time, source, engine, IP, attack type, severity, confidence, status, incident link.
- Filters: engine, severity, status, IP/CIDR, attack type, technique, time; saved views; bulk actions (mark false positive, merge into incident).
- Detail drawer: raw event, matched rule or model probabilities with top features, enrichment, neighbouring events for the same IP.

Page 4 - Incident Workbench

- Header: id, severity with score breakdown, status, SLA timers, owner.
- Tabs: Overview, Timeline, Evidence, Entities graph, Agent reasoning, Actions, Notes.
- **Timeline** shows every alert, enrichment, decision and action in order.
- **Entities graph** connects IPs, users, URLs, assets and files.
- Buttons: reverse action, extend block, reclassify, escalate, close as false positive, export report.

Page 5 - Case Board

- Kanban columns by status; cards show severity, asset, age and SLA; drag to change status; filters for owner and severity.
- Case page: summary written by the agent, tasks, comments, attachments, related incidents, closure form (root cause, lessons learned).

Page 6 - AI Agent Console

- Decision log with verdict, confidence, gate, tools used, tokens, latency and cost.

- Open any decision to read the evidence list and reasoning, and the exact prompt version used.
- Charts: verdict distribution, confidence histogram, agreement with analyst corrections, fallback rate, cost per day.
- Controls: autonomy mode (Autonomous, Shadow, Paused), gate thresholds (admin), prompt version selector, golden-set evaluation runs with results.

Page 7 - Autonomous Actions Ledger

- Every automatic action with target, tool, gate, result, verification status, expiry and rollback state.
- One-click **Undo** and **Extend**; filter by actor, tool and result; shows Policy Guard rejections and why.

Page 8 - Response and Blocklist

- Active blocks with remaining time, reason and incident; manual block and unblock (audited); allowlist management with owner and reason.
- Rate-limit and tarpit lists; HAProxy synchronisation status (file, live ACL, database) with a reconcile button.

15.4 Intelligence pages

Page 9 - Threat Intelligence Hub

- IP/domain/hash lookup with VirusTotal summary (engine counts, reputation, country, ASN, tags), AbuseIPDB score and our own history side by side.
- Quota meters for VirusTotal (per minute, per day) and cache hit rate; indicator table with confidence, freshness and expiry; STIX export.

Page 10 - MITRE ATT&CK Coverage

- Full ATT&CK Enterprise matrix coloured by detections in the selected period; separate colour for techniques that have rules but never fired and for techniques with no coverage.
- Click a technique to see its description, our rules, recent incidents and recommended mitigations; export a Navigator layer.

Page 11 - Playbook Studio

- List of playbooks with status, version, last run and success rate; YAML editor with validation; visual step graph; dry-run with sample incident; run history with per-step output and timing; enable, disable and roll back versions.

15.5 Pentest pages

Page 12 - Pentest Center

- Start a scan (target locked to the lab scope), choose profile (quick, standard, full), schedule recurring scans; live progress per tool.
- Run history; before/after comparison between website mode A and mode B; **detection test table** showing which launched attacks the SOAR detected and how fast.

Page 13 - Findings and Vulnerabilities

- Findings table with severity, CVSS, CWE, OWASP category, endpoint, status; detail with evidence and reproduction steps; link to remediation task and risk.
- Charts: findings by severity over time, mean time to fix, open versus fixed.

15.6 GRC pages

Page 14 - Asset Inventory

- Table and relationship view of assets with criticality, owner, exposure, last seen; per-asset panel with incidents, findings, risks and controls.

Page 15 - Risk Register and Heatmap

- 5x5 heatmap with risks plotted; register table with inherent and residual score, owner, treatment, due date; risk detail with linked findings and incidents; trend chart; acceptance workflow with expiry.

Page 16 - Compliance Center

- Gauges per framework (NIST CSF 2.0, ISO 27001, CIS v8, OWASP); per-function breakdown; control table with status, maturity, evidence count and review date; "biggest gaps" and "quickest wins" lists.

Page 17 - Policy Library

- Policies with version, owner, approval status, review date, acknowledgements; link to the playbook that enforces each policy; exceptions list with expiry.

Page 18 - Evidence and Audit

- Evidence browser filtered by control and period, with hash and source; tamper-evident audit log viewer with search by actor, action and target; export for auditors.

Page 19 - Reports

- Generate or schedule the executive and technical reports; preview; history; choose period and audience; download PDF.

15.7 Platform pages

Page 20 - Sources and Pipeline Health

- Events per second per source, last-seen time, stream lengths, consumer-group lag, dead-letter count, parsing error rate; alerts when a source goes silent.

Page 21 - Platform Health

- Service status, CPU and memory, container restarts, database sizes, backup status and last restore test, LLM provider status and latency, external API quotas.

Page 22 - Admin and RBAC

- Users, roles, API keys, notification routes, thresholds, allowlist, retention settings, configuration history with rollback.

Page 23 - Client Portal (read-only for Skyline)

- Security posture score, top risks, incidents summary in plain language, compliance percentage, downloadable reports, and the list of automatic actions taken on their behalf.

Page 24 - Ask Sentinel

- Natural-language questions such as "which IPs attacked login in the last hour" are translated into **predefined safe query templates** (not raw database commands), executed, and answered with a table and a short summary.

16. Platform Security and Access Control

Area	Requirement
Authentication	Password hashing with a modern algorithm, JWT access tokens with expiry, optional TOTP multi-factor, lockout on repeated failures
Roles	Admin, Analyst, Auditor (read-only), Client (read-only portal), Service accounts (per service, least privilege)
Authorisation	Role-based checks on every endpoint and every action tool; action-level permissions for autonomy settings
Secrets	Never in source control; environment or secret files; separate keys per service; rotation procedure documented
Audit	Append-only audit records for logins, configuration changes, playbook edits, agent decisions, actions, exports; hash-chained entries so tampering is detectable
Transport and storage	TLS between browser and API; encrypted backups; masking of personal data in logs
Application security	Input validation, CSRF and rate limits on the platform API, dependency scanning, container image scanning, non-root containers
Docker socket	The executor accesses Docker through a restricted proxy exposing only the needed calls
Separation of duties	The agent cannot change its own Policy Guard or allowlist; tuning proposals are applied by policy, not by the agent directly

17. Observability and Reliability

- Health endpoints on every service; heartbeat events to a status collection.
- Metrics: events per second, stream lengths and lag, detection latency, decision latency, action latency, LLM tokens and cost, cache hit rates, API quota use.
- Structured JSON logs with a correlation id that follows an event from ingestion to action.
- **Graceful degradation**: each dependency has a defined fallback.

Failure	Platform behaviour
LLM unavailable or slow	Fallback decision engine decides; decision marked fallback
VirusTotal quota exhausted	Continue with other sources; evidence notes the gap
Redis down	Sniffers buffer briefly on disk; services retry; dashboard shows degraded state
Detector crash	Consumer group redelivers unacknowledged events to another worker
HAProxy socket unreachable	Action marked failed, retried, then escalated; blocklist file still updated for next reload
Mongo down	Streams hold data; services retry with backoff

- Backup of MongoDB and configuration with scheduled **restore tests**; documented recovery time and recovery point targets.

18. Testing, Simulation and Validation

18.1 Test levels

- Unit tests: parsers, schema, scoring, rule evaluation, Policy Guard, tool validators.
- Integration tests: stream to detector to correlator to agent to executor with mocked LLM and mocked VirusTotal.
- End-to-end tests: real attacks from the pentest runner and an attacker container against website mode A.
- Security tests: prompt-injection suite, authorisation tests, scope-lock tests for the pentest runner.

18.2 Attack simulation scenarios (lab only)

Scenario	Expected platform reaction
Login brute force	Rule alert, incident, agent verdict, block with TTL, account flagged
HTTP flood	Stick-table 429 first, then ML/rate alert, tarpit, then block
SQL injection attempts	Signature alert, WAF rule, block, finding linked
Directory and port scanning	Scan rule, short block or tarpit
Malicious upload	Hash check, quarantine, block
Legitimate heavy user (false-positive test)	Agent returns false positive or uncertain; no block
Authorised pentest	Detected and labelled, scanner not blocked

18.3 Synthetic load test

A generator writes tens of thousands of events into the **events stream** (not directly into the database) to test the whole pipeline, the correlation reduction and the dashboard under load. It is documented as a **stress test, separate from detection accuracy**, with the metrics reported honestly (events per second, end-to-end latency, incidents produced).

18.4 Chaos and resilience tests

Stop Redis, stop the LLM, exhaust the VirusTotal quota, kill a detector, delay HAProxy; verify the fallbacks of section 17.

18.5 Acceptance criteria (examples)

Criterion	Target
Detection to block, for a clear brute-force attack	Under 15 seconds end to end
Wrong destructive actions on the golden set	0
Allowlisted address blocked	Never
Incident records contain evidence, reasoning and verification	100 percent
Dashboard live update delay	Under 3 seconds

19. Key Performance Indicators

KPI	Definition
MTTD	Attack start to first alert
MTTA	Alert to first acknowledgement (agent or human)
MTTC	Detection to containment executed and verified
MTTR	Detection to resolution
Autonomy rate	Incidents closed with no human action
Agent precision	Correct verdicts / all verdicts on the golden set
False-positive rate	Alerts or incidents later classified false positive
ATT&CK coverage	Techniques with at least one working detection / techniques relevant to the web stack
Pipeline throughput	Events per second sustained
Compliance percent	Per framework
Remediation time	Finding opened to retest-confirmed fix

20. Advanced Features That Make the Project Outstanding

Feature	Why it matters	Effort
Shadow mode for the agent	The agent decides but does not act for the first days; compare with analysts before enabling autonomy	Low
Attack storyline graph	One diagram per incident: recon, exploit, action, outcome	Medium
Attack replay	Record real attack traffic and replay it for demos and regression tests	Medium
ModSecurity with the OWASP Core Rule Set	A real WAF as an extra detection and enforcement source	Medium
Second network sensor (Suricata)	Signature-based alerts that corroborate the ML model	Medium
Deception endpoints (honeypot paths)	Almost zero false positives; instant high-confidence alerts	Low
Purple-team automation (Atomic Red Team style)	Measures ATT&CK coverage automatically	Medium
Host and container logs (Wazuh agent)	Adds endpoint-level visibility	Medium
Image and dependency scanning (Trivy)	Feeds vulnerabilities into GRC	Low
Auto-tuning with the Tuning Agent	Proposes thresholds and allowlist changes from feedback, with canary rollout	Medium
Model drift monitor and retraining pipeline	Keeps the ML model honest	Medium
Multi-tenant mode with a second fictional customer	Proves the MSSP design; strict data separation	High
Ask Sentinel natural-language search	Analyst productivity; impressive demo	Medium
Business-impact calculator	Converts downtime into estimated money for executives	Low
STIX/TAXII export of indicators	Standards-based sharing	Low
Agent cost and latency dashboard	Shows engineering maturity	Low
Backup and recovery drill	Covers the Recover function of NIST CSF	Low
Mobile-friendly client portal	Executives can view posture on a phone	Low

21. Coverage Against an Enterprise SOAR Reference, Opinion, and Sources

21.1 Coverage of the 30-module enterprise reference architecture

The team prepared a 30-module reference list drawn from public Splunk SOAR and Cortex XSOAR documentation. This table shows where each module is covered. **Core** means built for the demo; **Included** means built in a reduced form; **Roadmap** means described but not required.

Module	Where covered	Level
1 Data ingestion	Sections 3, 4	Core
2 Integration and connectors	Sections 7, 10 (connector pattern)	Core
3 Alert management and triage	Sections 5, 6, 9	Core
4 Correlation engine	Section 6	Core
5 Incident and case management	Section 11	Core
6 Playbook engine	Section 8	Core
7 Workflow orchestration	Sections 8, 10	Included
8 Automation execution	Section 10	Core
9 Investigation and evidence	Sections 9.1, 13.5, 15	Included
10 Threat intelligence	Section 7	Core
11 Threat hunting	Search tools, Ask Sentinel	Roadmap
12 Detection engineering	Sections 5, 20	Included
13 AI and machine learning	Sections 5.2, 20	Core
14 LLM engine and agents	Section 9	Core
15 Response and containment	Section 10	Core
16 Human approval and analyst loop	Review, undo and shadow mode (post-hoc by design)	Included
17 Asset management	Section 13.1	Core
18 Identity context	Account lock and auth events	Included
19 Vulnerability and exposure	Section 12, 13	Core
20 Cloud and container security	Container isolation, Trivy	Roadmap
21 Risk and business impact	Section 13	Core
22 Dashboards and reporting	Section 15	Core
23 Security, governance, compliance	Sections 13, 16	Core
24 Platform infrastructure	Sections 2, 17	Core
25 Administration and APIs	Sections 15 (Admin), 4	Included
26 Knowledge base and runbooks	Playbooks, policies, agent retrieval of policies	Included
27 Notification and escalation	Section 14	Core
28 Testing and simulation	Section 18	Core
29 Continuous improvement	Tuning Agent, feedback	Included
30 Advanced enterprise and MSSP	Multi-tenant, client portal	Roadmap

21.2 Honest engineering notes

- **Autonomy is the headline feature and the main risk.** Automatic blocking is acceptable because each action is time-limited, verified and reversible and the Policy Guard is code. Keep Shadow Mode and the Kill Switch; the committee will ask about them.
- **LLM quality and cost**: run the agent only on correlated incidents, cache enrichment, and keep the deterministic fallback.
- **Free-tier limits**: the VirusTotal public key (about 4 requests per minute and 500 per day) is enough for a demo only with caching and the quota governor.
- **No public source proves that any list is literally every component of the world's largest SOAR**; the 30-module list is a reference architecture, not a vendor claim.

- **Honest reporting**: the stress test shows platform capacity; the detection accuracy comes from labelled data and live attack scenarios; never mix the two in the report.

21.3 Suggested demo script

- Show the healthy platform and the client portal.
- Run a pentest scan on website mode A; show findings appearing and risks updating.
- Launch a brute-force attack from the attacker container; watch alert, incident, agent reasoning, automatic block and notification within seconds.
- Show the actions ledger, undo the block, and show the audit entry.
- Send a legitimate heavy-use pattern; show the agent refusing to block.
- Switch to website mode B, rerun the scan, show the before/after report and the compliance percentage change.
- Download the executive report.

21.4 Sources

- Splunk SOAR documentation: help.splunk.com/en/splunk-soar
- Cortex XSOAR administrator guide (Palo Alto Networks documentation)
- VirusTotal API v3 documentation; public API limits of about 4 requests per minute and 500 per day
- HAProxy Runtime API reference (add acl, del acl, show acl, clear acl, prepare/commit acl)
- MITRE ATT&CK: attack.mitre.org (v19, April 2026) and the attack-stix-data repository (STIX 2.1)
- NIST CSF 2.0, ISO/IEC 27001:2022, CIS Controls v8, OWASP Top 10